

Plant Identification

Refer to the *Sys_iden_block.slx* and *Sys_iden_script.m* files, as well as the notes for the circuit diagram to be implemented for system identification.

Task A



For the given input parameters, with initial calculated K_i and K_p , as in notes:

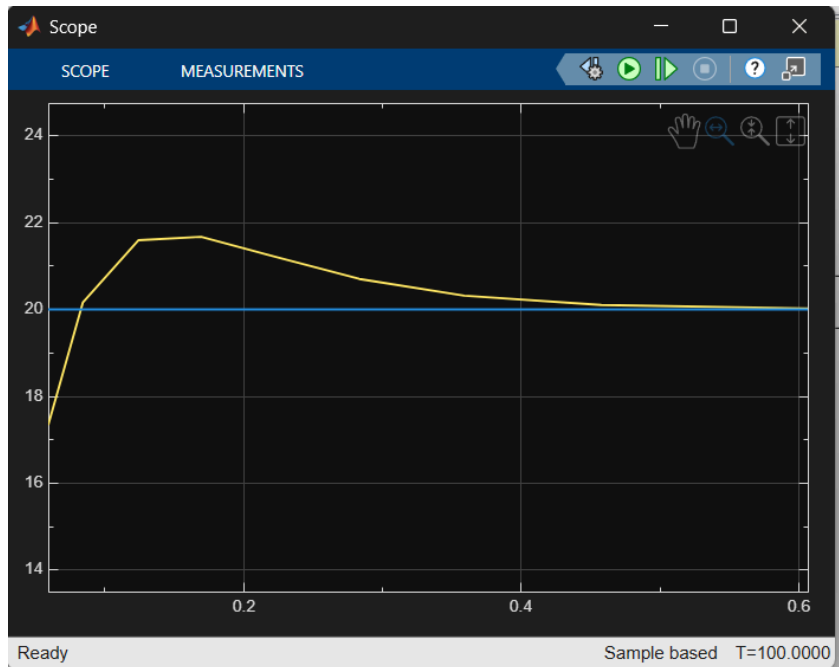
The overshoot is under 10%

No steady state error

To reduce settling time under 0.5s, since i need to increase freq, which is directly related to K_i , K_i is increased

But increasing freq, reduces damping ratio as the K_p term related to alpha stays constant, so K_p also needs to be increased to reduced overshoot that would otherwise happen

With new K_i and K_p values as found in the notes:



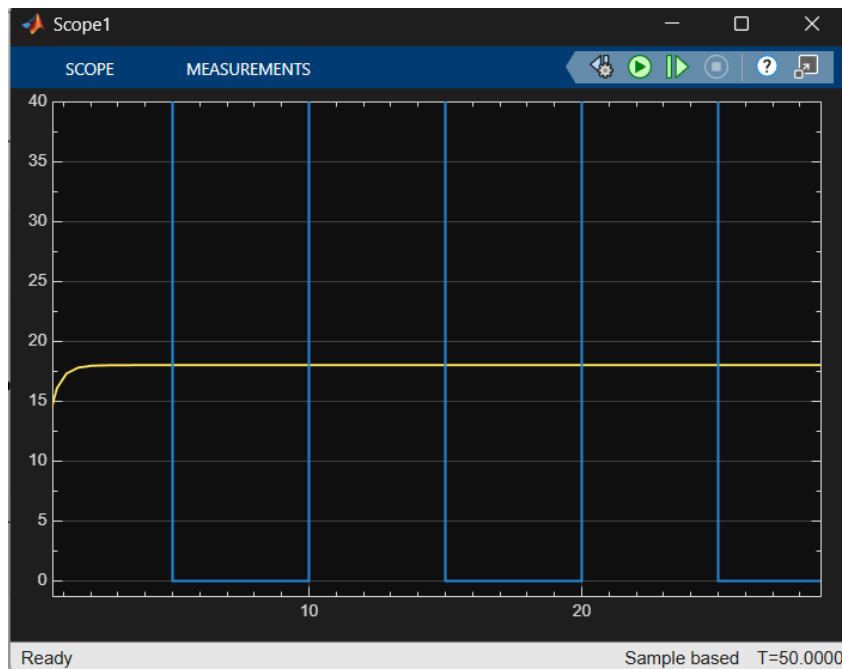
All targets achieved

Task B



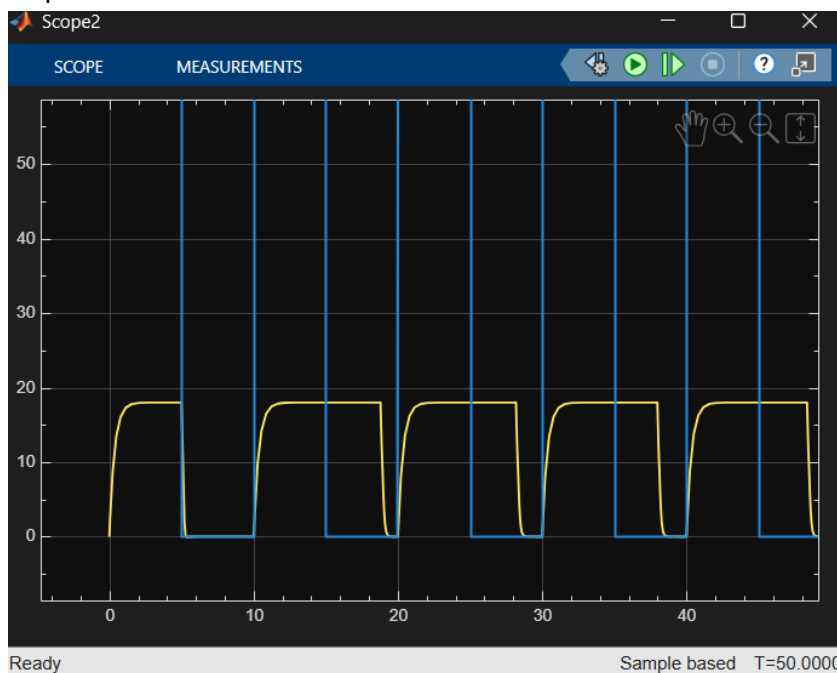
Due the the physical limitations of the plant model, it cannot reach the absurd 200rad/s command, even though the integral term is massive, it doesnt show

To see the effect, we can use a pulse generator to see how the massive integral term wont allow sudden change from 200 target



The motor is stuck at the max of 18, when it should have dropped, this is because of the massive integral term, pulling it towards 200 from the initial high pulse

Anti windup fix: Using clamping, which is freezing the integral term from adding error, when output PI controller is more than saturation limit set as ± 12 .



The windup fix has helped reduce, the windup for the first 10 seconds, but for some reason it holds 18rad/s beyond 15s even though the pulse is low, and since integral term is clamped and

the K_p value is 6.33, so the error would be 6.33×18 , which should push it back to zero. Unable to debug, tried with back calculation method in sumlink, problem persists

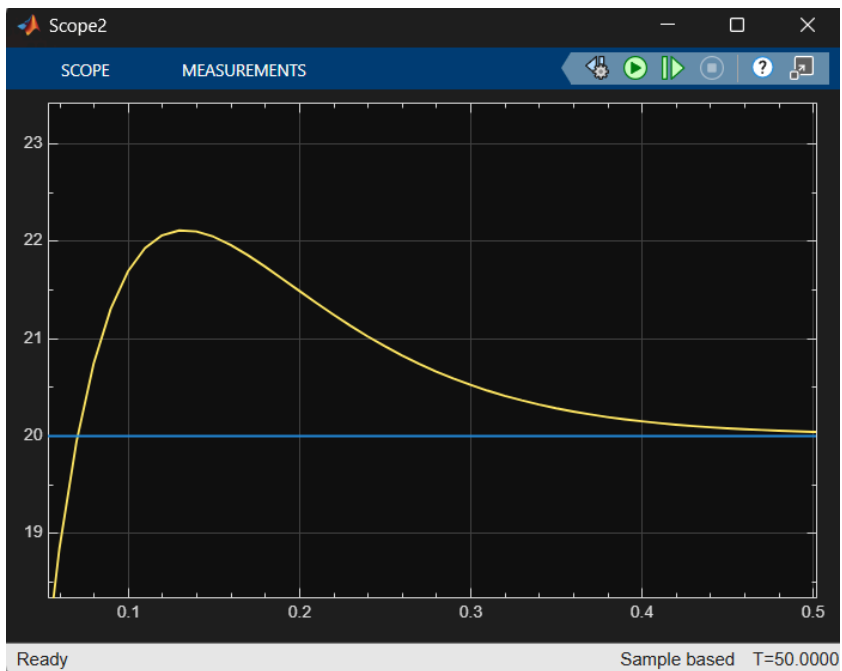
Task C

For 1kHz sampling time = 0.001s



No real change from continuous time domain plot

For sampling time = 0.01s, the overshoot has considerably gone up



For sampling time = 0.05s, even more over shoot, but lesser settling time



This is happening because the time the controller takes to process and give output, the system has evolved, basically reacting to the past now, so this introduces a time delay, which reduces the phase margin, when phase margin drops, the damping reduces

Phase margin: In a negative feedback loop we have a leverage that the output of the system can be upto 180deg late to the input signal, once it reaches 180, since we are taking negative feedback, we end up inverting the 180deg late output and thus the controller becomes positive feedback, so it speeds up instead of slowing down

The output is delayed from the input due to lag of electrical components like inductors, and mechanical inertia of the shaft and load

Margin is how much we have left before we hit the 180deg mark

Since margin is tied to damping, in an approximately direct relationship, as the margin decreases, damping decreases, therefore overshoot increases.

The acceptable freq of controller based on task A requirement is found to be 200Hz based on trial and error:



Task D

Increasing Prediction horizon increases how far ahead the MPC looks ahead, need to balance between making it too short sighted vs too much computation

Increasing Control horizon allows controller to plan more ahead, but needs more computation, Decreasing it makes the action more slow but smoother

Once the controller finds suppose N_c amount of steps to take, it only performs one of those and throws away the rest (if it executed those 3, it would be blind to changes in the env between them, why not just use $N_c=1$, because that limits the algo to think you can only perform one step, limiting how complex of a procedure it can plan to execute the goal)

$$\text{Total Penalty Score} = \sum (\text{OV_Weight} \times \text{Speed Error}^2) + \sum (\text{MVRate_Weight} \times \Delta \text{Voltage}^2)$$

Here OV is the output variable error you are willing to bear

MV rate is the rate at which you are willing to push your supply

Running MPC for the same input setup as taskB to see how it actually performs, as the motor plant caps out at 18rad/s



This is for $OV=1$ (I hate error in my output speed)
 $MVrate=0.1$ (I can tolerate some level of spiking my input source)

Task E

Clearly from the scope outputs, MPC is much better equipped to handle this control as it handles the input proactively instead of reactively

However this added complexity of the MPC model comes at computation cost

Running Task_E_script gives the following output:

```
>> Task_E_script
Running PID Simulation...
Running MPC Simulation...
-----
PID Compute Time: 1.146461 seconds
MPC Compute Time: 13.493195 seconds
MPC was 11.8 times slower to compute!
```

A massive time difference

Clearly, If the PI loop is able to achieve decent perform for much lower computational cost For a first order plant model, where i dont have much to consider, PID tuning is more cost effective and viable

However if I needed a control system for a much more complex and vital system, like object avoidance and path planning, the foresight of a MPC is much more useful and even required

